Django Database & ORM — Complete Notes A to Z

Django 6.0 | Beginner to Advanced | Simple English

Table of Contents

1. Database Basics
2. Django ORM
3. Project Setup
4. Django Models and Fields
5. Migrations
6. CRUD Operations
7. QuerySets and Filtering
8. Field Lookups and Q Objects
9. Model Relationships
10. Ordering and Limiting Results
11. select_related and prefetch_related
12. Aggregation and Annotation
13. F Expressions
14. Conditional Expressions
15. Subquery and Exists
16. Managers and Custom QuerySets
17. Database Transactions
18. Raw SQL
19. Query Optimization and Indexes
20. Bulk Operations
21. Multiple Databases
22. Fixtures
23. Signals
24. Validation and Security
25. Interview Questions
26. Quick Cheat Sheet
27. Learning Roadmap

Chapter 1: Database Basics

1.1 What Is a Database?

A database is a place where we store and manage data.

Examples:

- Student names and emails
- Product information
- Customer orders
- User accounts
- Examination results

Popular databases:

- PostgreSQL
- MySQL
- SQLite
- Oracle

1.2 What Is a Relational Database?

A relational database stores data in tables.

Term	**Simple Meaning**
Table	A collection of data
Row	One record
Column	One field
Primary Key	Uniquely identifies a record
Foreign Key	Connects two tables

Constraint	A rule for data
Index	Helps find data faster

Example:

id	name	email
1	Rahim	rahim@example.com
2	Karim	karim@example.com

Here, `id` can be the primary key.

1.3 What Is SQL?

SQL means Structured Query Language.

We use SQL to work with relational databases.

```
SELECT * FROM students;

SELECT * FROM students
WHERE age >= 18;

INSERT INTO students (name, age)
VALUES ('Rahim', 20);

UPDATE students
SET age = 21
WHERE id = 1;
```

```
DELETE FROM students
WHERE id = 1;
```

1.4 What Is ACID?

ACID describes important database transaction properties.

- **Atomicity:** All operations succeed, or the transaction is rolled back.
- **Consistency:** Data follows database rules.
- **Isolation:** Concurrent transactions follow the database's isolation rules.
- **Durability:** Committed changes are saved according to the database's guarantees.

Chapter 2: Django ORM

2.1 What Is ORM?

ORM means Object-Relational Mapper.

Django ORM lets us work with a database using Python code instead of writing SQL for every query.

SQL:

```
SELECT * FROM students WHERE age >= 18;
```

Django ORM:

```
Student.objects.filter(age__gte=18)
```

2.2 Benefits of Django ORM

- Write database queries using Python.
- Manage database tables through models and migrations.
- Work with related tables.
- Build queries by combining QuerySets.
- Use parameterized queries for many database operations.
- Change database backends more easily, although some features differ between databases.

2.3 View the SQL Query

```
students = Student.objects.filter(age__gte=18)

print(students.query)
```

Understanding SQL is still useful for debugging and improving performance.

Chapter 3: Project Setup

3.1 Install Django

Create a virtual environment:

```
python -m venv .venv
```

Activate it on Windows:

```
.venv\Scripts\activate
```

Activate it on macOS or Linux:

```
source .venv/bin/activate
```

Install Django 6.0:

```
python -m pip install "Django>=6.0,<6.1"
```

Create a project:

```
django-admin startproject config .
```

Create an app:

```
python manage.py startapp students
```

Check the installed version:

```
python -m django --version
```

3.2 Register the App

Open `config/settings.py`.

Add your app to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Default Django apps
    "students",
]
```

3.3 SQLite Configuration

A new Django project normally uses SQLite by default.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

SQLite is useful for learning and many small projects.

3.4 PostgreSQL Configuration

Install a PostgreSQL driver that is compatible with your environment.

You can read database settings from environment variables:

```
import os

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "localhost"),
        "PORT": os.environ.get("DB_PORT", "5432"),
    }
}
```

Security tip: Never hard-code production passwords in your source code or commit them to Git.

Chapter 4: Django Models and Fields

4.1 What Is a Model?

A model is a Python class that describes the structure of data stored in a database table.

Example: `students/models.py`

```
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField(default=18)
```

```
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Django normally creates a primary key field automatically if you do not define one.

4.2 Common Model Fields

```
models.CharField(max_length=100)
models.TextField()
models.IntegerField()
models.PositiveIntegerField()
models.DecimalField(max_digits=10, decimal_places=2)
models.FloatField()
models.BooleanField(default=True)
models.DateField()
models.DateTimeField()
models.EmailField()
models.URLField()
models.UUIDField()
models.JSONField()
models.FileField(upload_to="files/")
models.ImageField(upload_to="images/")
```

Use `DecimalField` for money when decimal precision is important.

4.3 null=True vs blank=True

```
middle_name = models.CharField(
    max_length=100,
    blank=True,
)
```

- `blank=True`: The field may be empty during validation.
- `null=True`: The database can store SQL `NULL`.

For string fields, Django projects usually prefer an empty string instead of using both `NULL` and an empty string.

4.4 default, unique, and db_index

```
status = models.CharField(
    max_length=20,
```

```
    default="pending",
)

email = models.EmailField(unique=True)

slug = models.SlugField(db_index=True)
```

- `default` : Sets a default value.
- `unique=True` : Prevents duplicate values at the database level.
- `db_index=True` : Adds a database index.

Indexes can make some queries faster, but they also use storage and can slow down writes.

4.5 Meta Options

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["name"]
        indexes = [
            models.Index(fields=["created_at"]),
        ]
        verbose_name = "Student"
        verbose_name_plural = "Students"
```

`ordering` defines the default order of query results.

4.6 Model Methods

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    def __str__(self):
        return self.name

    def is_expensive(self):
        return self.price >= 1000
```

4.7 Choices

```
class Order(models.Model):

    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        PAID = "paid", "Paid"
        CANCELLED = "cancelled", "Cancelled"

    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
```

Choices help with validation and display. Use database constraints when you need a database-level rule.

Chapter 5: Migrations

5.1 What Is a Migration?

A migration describes a change to the database structure.

For example, you add a new field to a model. A migration can apply that change to the database.

5.2 Important Commands

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate students 0001
```

Command	Purpose
`makemigrations`	Creates migration files

`migrate`	Applies migrations to the database
`showmigrations`	Shows migration status
`sqlmigrate`	Shows the SQL for a migration

5.3 Migration Workflow

1. Change `models.py`.
2. Run `makemigrations`.
3. Review the migration file.
4. Run `migrate`.
5. Run your tests.

Keep migration files in version control.

Before changing a production database, consider backups, deployment plans, and rollback strategies.

Chapter 6: CRUD Operations

CRUD means Create, Read, Update, and Delete.

6.1 Create

```
student = Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
    age=20,
)
```

Another way:

```
student = Student(
    name="Karim",
    email="karim@example.com",
    age=21,
)

student.save()
```

6.2 Read

```
Student.objects.all()

Student.objects.get(pk=1)

Student.objects.filter(is_active=True)

Student.objects.exclude(age__lt=18)
```

`get()` expects exactly one object.

It raises `DoesNotExist` if no object matches and `MultipleObjectsReturned` if more than one object matches.

```
try:
    student = Student.objects.get(pk=1)
except Student.DoesNotExist:
    student = None
```

6.3 Update One Object

```
student = Student.objects.get(pk=1)

student.age = 22

student.save(update_fields=["age"])
```

6.4 Update Multiple Objects

```
Student.objects.filter(
    is_active=False
).update(is_active=True)
```

`QuerySet.update()` updates matching rows directly. It does not call each object's `save()` method or normal save signals.

6.5 Delete

```
student = Student.objects.get(pk=1)

student.delete()
```

Delete multiple objects:

```
Student.objects.filter(is_active=False).delete()
```

Always check your filter before deleting data.

6.6 get_or_create()

```
student, created = Student.objects.get_or_create(
    email="rahim@example.com",
    defaults={
        "name": "Rahim",
        "age": 20,
    },
)
```

It finds a matching object or creates one if none exists.

6.7 update_or_create()

```
student, created = Student.objects.update_or_create(
    email="rahim@example.com",
    defaults={
        "name": "Rahim Updated",
        "age": 21,
    },
)
```

Database uniqueness constraints are important when concurrent requests may create the same record.

Chapter 7: QuerySets and Filtering

7.1 What Is a QuerySet?

A QuerySet represents a database query and its results.

```
students = Student.objects.filter(age__gte=18)
```

QuerySets are usually lazy. Creating one does not normally execute the database query immediately.

```
students_list = list(students)
```

This evaluates the QuerySet.

7.2 QuerySet Chaining

```
students = (
    Student.objects
    .filter(is_active=True)
    .filter(age__gte=18)
    .order_by("name")
)
```

7.3 Common Filters

```
Student.objects.filter(name="Rahim")

Student.objects.filter(age__gte=18)

Student.objects.filter(age__range=(18, 25))

Student.objects.filter(name__startswith="R")

Student.objects.filter(
    email__icontains="example.com"
)

Student.objects.filter(
    created_at__isnull=False
)
```

7.4 QuerySet Evaluation

Common operations that may evaluate a QuerySet include:

```
list(students)
len(students)
bool(students)

for student in students:
    print(student.name)
```

An evaluated QuerySet can cache its results. A new QuerySet does not automatically share that cache.

Chapter 8: Field Lookups and Q Objects

8.1 Common Lookups

Lookup	Meaning
`exact`	Exact match
`iexact`	Case-insensitive exact match
`contains`	Contains text
`icontains`	Case-insensitive contains
`gt`	Greater than
`gte`	Greater than or equal
`lt`	Less than
`lte`	Less than or equal
`in`	Value is in a list

`range`	Value is in a range
`isnull`	Checks for NULL
`startswith`	Starts with text
`endswith`	Ends with text

Examples:

```
Student.objects.filter(age__gte=18)

Student.objects.filter(id__in=[1, 2, 3])

Student.objects.filter(name__icontains="rah")
```

8.2 Q Objects

Q objects help create complex conditions.

```
from django.db.models import Q
```

OR condition:

```
students = Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)
```

AND condition:

```
students = Student.objects.filter(
    Q(is_active=True) & Q(age__gte=18)
)
```

NOT condition:

```
students = Student.objects.filter(
    ~Q(name__icontains="test")
)
```

Normal filter arguments are combined using AND.

Chapter 9: Model Relationships

9.1 ForeignKey — One-to-Many

One author can have many posts.

```
class Author(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name


class Post(models.Model):
    author = models.ForeignKey(
        Author,
        on_delete=models.CASCADE,
        related_name="posts",
    )

    title = models.CharField(max_length=200)
    body = models.TextField()
```

Usage:

```
author = Author.objects.get(pk=1)

author.posts.all()

post = Post.objects.get(pk=1)

post.author
```

on_delete Options

- `CASCADE`: Related objects are deleted when the referenced object is deleted.
- `PROTECT`: Prevents deletion when protected related objects exist.
- `RESTRICT`: Restricts deletion under certain relationship conditions.
- `SET_NULL`: Sets the foreign key to NULL. The field needs `null=True`.
- `SET_DEFAULT`: Sets the foreign key to its default value.
- `DO_NOTHING`: Django takes no automatic action; the database may raise an error.

9.2 OneToOneField

```
class Profile(models.Model):
    student = models.OneToOneField(
        Student,
        on_delete=models.CASCADE,
        related_name="profile",
    )

    bio = models.TextField(blank=True)
```

Each student can have at most one profile through this relationship.

9.3 ManyToManyField

A student can take many courses, and a course can have many students.

```
class Course(models.Model):
    title = models.CharField(max_length=100)


class Learner(models.Model):
    name = models.CharField(max_length=100)

    courses = models.ManyToManyField(
        Course,
        related_name="learners",
        blank=True,
    )
```

Usage:

```
learner = Learner.objects.get(pk=1)
course = Course.objects.get(pk=1)

learner.courses.add(course)

learner.courses.remove(course)

learner.courses.set([course])

learner.courses.clear()
```

9.4 Through Model

Use a through model when a relationship needs extra information.

```
class Enrollment(models.Model):
    learner = models.ForeignKey(
        Learner,
        on_delete=models.CASCADE,
    )

    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
    )

    enrolled_at = models.DateTimeField(
        auto_now_add=True
    )

    grade = models.CharField(
        max_length=5,
        blank=True,
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["learner", "course"],
                name="unique_learner_course",
            )
        ]
```

You can connect this model to a `ManyToManyField` using `through="Enrollment"`.

Chapter 10: Ordering and Limiting Results

10.1 Ordering

```
Student.objects.order_by("name")

Student.objects.order_by("-created_at")

Student.objects.order_by("name", "-age")
```

A minus sign means descending order.

10.2 Limiting Results

```
Student.objects.all()[:10]

Student.objects.all()[10:20]
```

These slices normally use SQL `LIMIT` and `OFFSET`.

10.3 values()

```
Student.objects.values("id", "name", "email")
```

This returns dictionaries containing the selected fields.

10.4 values_list()

```
Student.objects.values_list("name", flat=True)
```

This returns a list-like result containing the names.

10.5 exists()

```
if Student.objects.filter(is_active=True).exists():
    print("Active students exist")
```

Use `exists()` when you only need to know whether matching records exist.

10.6 count()

```
total_students = Student.objects.count()
```

10.7 distinct()

```
Post.objects.values("author_id").distinct()
```

`distinct()` removes duplicate results. Some distinct features depend on the database backend.

Chapter 11: select_related and prefetch_related

These methods can reduce the N+1 query problem.

11.1 N+1 Query Problem

```
posts = Post.objects.all()

for post in posts:
    print(post.author.name)
```

If Django fetches each author separately, the number of queries may grow quickly.

11.2 select_related()

`select_related()` uses SQL JOINs for relationships such as ForeignKey and OneToOneField.

```
posts = Post.objects.select_related("author")

for post in posts:
    print(post.author.name)
```

11.3 prefetch_related()

`prefetch_related()` normally uses additional queries to load related collections.

```
authors = Author.objects.prefetch_related("posts")

for author in authors:
    for post in author.posts.all():
        print(post.title)
```

You can combine both methods:

```
Post.objects.select_related(
    "author"
).prefetch_related("tags")
```

Choose the method based on your relationships and measured query performance.

Chapter 12: Aggregation and Annotation

```
from django.db.models import Avg, Count, Max, Min, Sum
```

12.1 aggregate()

`aggregate()` returns a summary for the whole QuerySet.

```
Student.objects.aggregate(
    average_age=Avg("age"),
    oldest=Max("age"),
    youngest=Min("age"),
)
```

The result is a dictionary.

12.2 annotate()

`annotate()` adds a calculated value to each returned object.

```
authors = Author.objects.annotate(
    post_count=Count("posts")
)

for author in authors:
    print(author.name, author.post_count)
```

12.3 Filtering by Annotation

```
authors = (
    Author.objects
    .annotate(post_count=Count("posts"))
    .filter(post_count__gte=5)
)
```

Some joins can create duplicate rows. In those cases, `Count(..., distinct=True)` may be needed.

Chapter 13: F Expressions

An `F()` expression refers to a database field inside a query.

```
from django.db.models import F
```

Reduce product stock:

```
Product.objects.filter(
    stock__gt=0
).update(
    stock=F("stock") - 1
)
```

The calculation happens in the database instead of first reading the old value into Python.

Compare fields:

```
Employee.objects.filter(
    salary__gt=F("manager__salary")
)
```

After saving an `F()` expression on a model instance, refresh the instance before using it as an ordinary Python value.

```
product = Product.objects.get(pk=1)

product.stock = F("stock") - 1

product.save(update_fields=["stock"])

product.refresh_from_db()
```

Chapter 14: Conditional Expressions

14.1 Case and When

You can calculate values based on conditions.

```
from django.db.models import Case, When, Value, CharField

students = Student.objects.annotate(
    age_group=Case(
        When(age__lt=18, then=Value("minor")),
        default=Value("adult"),
        output_field=CharField(),
    )
)
```

14.2 Database Functions

```
from django.db.models.functions import Lower, Upper, Length
```

Examples:

```
Student.objects.annotate(
    lower_name=Lower("name")
)

Student.objects.annotate(
    upper_name=Upper("name")
)

Student.objects.annotate(
    name_length=Length("name")
)
```

14.3 Coalesce

`Coalesce` returns the first non-NULL expression.

```
from django.db.models.functions import Coalesce
```

The fallback value must have a compatible type.

Chapter 15: Subquery and Exists

15.1 Exists

Use `Exists` to check whether related records exist.

```
from django.db.models import Exists, OuterRef

posts = Post.objects.filter(
    author=OuterRef("pk")
)

authors = Author.objects.annotate(
    has_posts=Exists(posts)
)
```

15.2 Subquery

Get a related post title for each author:

```
from django.db.models import OuterRef, Subquery

latest_post = (
    Post.objects
    .filter(author=OuterRef("pk"))
    .order_by("-pk")
)

authors = Author.objects.annotate(
    latest_post_title=Subquery(
        latest_post.values("title")[:1]
    )
)
```

For real projects, use a suitable timestamp such as `created_at` when deciding which post is the latest. Make sure a subquery returns the number of rows and columns required by the expression.

Chapter 16: Managers and Custom QuerySets

A Manager provides an interface for accessing a model's database queries.

```
class PublishedPostQuerySet(models.QuerySet):

    def published(self):
        return self.filter(is_published=True)

    def by_author(self, author):
        return self.filter(author=author)


class Post(models.Model):
    title = models.CharField(max_length=200)

    is_published = models.BooleanField(
        default=False
    )

    objects = PublishedPostQuerySet.as_manager()
```

Usage:

```
Post.objects.published()

Post.objects.published().by_author(author)
```

Be careful when adding default filters to a Manager. They can hide records from admin pages or other workflows.

Chapter 17: Database Transactions

A transaction groups database operations into one logical unit.

17.1 transaction.atomic()

```
from django.db import transaction

with transaction.atomic():
    order = Order.objects.create(...)

    OrderItem.objects.create(
        order=order,
        ...
    )
```

If an exception causes the transaction to fail, its database changes can be rolled back.

17.2 Why Use Transactions?

Suppose you need to:

1. Create an order.
2. Create order items.
3. Update inventory.

If an important operation fails, you may not want to keep only part of the changes. A transaction can help keep the database consistent.

17.3 on_commit()

Run a callback after a successful transaction commit:

```
from django.db import transaction

transaction.on_commit(
    lambda: print("Transaction committed")
)
```

For emails and external API calls, timing matters. Reliable background processing may require a task queue or an outbox pattern.

Chapter 18: Raw SQL

Use the ORM for most queries. Raw SQL can be useful for special or complex queries.

18.1 raw()

```
students = Student.objects.raw(
    """
    SELECT *
    FROM students_student
    WHERE age >= %s
    """,
    [18],
)
```

The table name must match the actual database table name. A raw model query normally needs the model's primary key in the selected columns.

18.2 Cursor

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        """
        SELECT COUNT(*)
        FROM students_student
        WHERE age >= %s
        """,
        [18],
    )

    row = cursor.fetchone()

print(row[0])
```

Security: Never join user input directly into an SQL string. Use parameterized queries.

Chapter 19: Query Optimization and Indexes

19.1 Query Plan

```
students = Student.objects.filter(age__gte=18)

print(students.explain())
```

`explain()` helps you understand how the database plans to execute a query.

19.2 When Should You Use Indexes?

Indexes can help with:

- Frequently filtered fields
- Join keys
- Frequently ordered fields
- Common combinations of fields used by queries

Do not add indexes to every field. Indexes use storage and can slow down writes.

19.3 Composite Index

```
class Meta:
    indexes = [
        models.Index(
            fields=["is_active", "created_at"]
```

```
        ),
    ]
```

The order of fields in a composite index matters.

19.4 Optimization Checklist

- Reduce N+1 queries.
- Use `select_related()` and `prefetch_related()`.
- Load only the fields you need.
- Use pagination for large result sets.
- Measure query counts and response times.
- Check query plans.
- Add indexes based on real query patterns.

Chapter 20: Bulk Operations

Bulk operations can process many records efficiently.

20.1 bulk_create()

```
students = [
    Student(
        name="A",
        email="a@example.com",
    ),
    Student(
        name="B",
        email="b@example.com",
    ),
]

Student.objects.bulk_create(
    students,
    batch_size=500,
)
```

20.2 bulk_update()

```
students = list(
    Student.objects.filter(is_active=False)
)
```

```
for student in students:
    student.is_active = True

Student.objects.bulk_update(
    students,
    ["is_active"],
    batch_size=500,
)
```

Bulk operations generally do not call each model instance's `save()` method or normal save signals. Backend behavior can vary.

Chapter 21: Multiple Databases

Django can connect to multiple databases.

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    },
    "analytics": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "analytics.sqlite3",
    },
}
```

Read from a specific database:

```
Student.objects.using("analytics").all()
```

Save to a specific database:

```
student.save(using="analytics")
```

Multiple databases may require database routers and migration planning. Cross-database relationships and transactions have limitations.

Chapter 22: Fixtures

Fixtures are files that contain data for loading into a database.

Export data:

```
python manage.py dumpdata students --indent 2 > students.json
```

Import data:

```
python manage.py loaddata students.json
```

Consider primary keys, model dependencies, and environment-specific data when using fixtures.

Remove private information before sharing data exports.

Chapter 23: Signals

Signals let a function respond to certain events.

For example, `post_save` runs after a model instance is saved.

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Student


@receiver(post_save, sender=Student)
def student_saved(
    sender,
    instance,
    created,
    **kwargs
):
    if created:
        print(
            f"New student created: {instance.pk}"
        )
```

You can register signal handlers by importing the signal module from your app configuration's `ready()` method.

Signal Best Practices

- Avoid hiding complex business logic in signals.
- `QuerySet.update()` does not call normal save signals.
- Bulk operations generally do not call per-instance save signals.
- Avoid registering the same receiver more than once.
- Explicit service functions can be easier to understand and test.

Chapter 24: Validation, Constraints and Security

24.1 Model Validation

```
student = Student(
    name="",
    email="invalid-email",
)

student.full_clean()
```

Django's `save()` method does not automatically call `full_clean()`.

24.2 CheckConstraint

Example: Product price must not be negative.

```
class Product(models.Model):
    name = models.CharField(max_length=100)

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(price__gte=0),
                name="product_price_nonnegative",
            )
        ]
```

Check the official documentation for the correct API for your Django version.

24.3 UniqueConstraint

```
class Enrollment(models.Model):
    learner = models.ForeignKey(
        "Learner",
        on_delete=models.CASCADE,
    )

    course = models.ForeignKey(
        "Course",
        on_delete=models.CASCADE,
    )
```

```
    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["learner", "course"],
                name="unique_enrollment",
            )
        ]
```

A database constraint helps prevent duplicate records even when concurrent requests happen.

24.4 Security Checklist

- Use parameterized queries for raw SQL.
- Check user permissions.
- Keep credentials out of source code.
- Set `DEBUG=False` in production.
- Configure `ALLOWED_HOSTS` correctly.
- Do not log sensitive information.
- Give database users only the permissions they need.
- Test your backup and restore process.
- Allowlist user-selectable field names and ordering options.

Chapter 25: Django ORM Interview Questions

1. What is Django ORM?

It lets you work with a relational database through Python models and query methods.

2. What is a Model?

A Python class that defines a database table's structure and behavior.

3. What is a Migration?

A versioned instruction for changing the database schema.

4. What is the difference between makemigrations and migrate?

`makemigrations` creates migration files. `migrate` applies migrations to the database.

5. What is the difference between get() and filter()?

`get()` expects one object and can raise an exception. `filter()` returns a QuerySet.

6. What does lazy QuerySet mean?

A QuerySet usually does not run its database query until its results are needed.

7. What is the difference between select_related() and prefetch_related()?

`select_related()` uses SQL JOINs for ForeignKey and OneToOne relationships. `prefetch_related()` normally uses additional queries to load related collections.

8. What is the difference between annotate() and aggregate()?

`annotate()` adds calculated values to each object. `aggregate()` returns a summary for the whole QuerySet.

9. What is an F expression?

It refers to a database field inside a query and allows database-side calculations.

10. What is a Q object?

It creates complex query conditions such as OR, AND, and NOT.

11. What is the difference between null=True and blank=True?

`null` affects database storage. `blank` affects validation.

12. What does CASCADE do?

It causes related objects to be deleted when the referenced object is deleted.

13. Should every field have an index?

No. Indexes improve some reads but use storage and can slow down writes.

14. What is transaction.atomic()?

It groups database operations into a transaction that can roll back when an error occurs.

15. What is the difference between update() and save()?

`update()` changes matching rows directly. `save()` saves a model instance.

16. What is the N+1 query problem?

It happens when related data causes an additional query for many individual records.

17. Why use bulk_create()?

It can insert many records efficiently, but it does not provide all normal per-instance save behavior.

18. Why use database constraints?

They protect data integrity at the database level.

19. When should you use exists()?

When you only need to know whether matching records exist.

20. How do you optimize ORM queries?

Measure query counts, inspect query plans, reduce unnecessary queries, use indexes carefully, and load only the data you need.

Chapter 26: Quick ORM Cheat Sheet

```
# CREATE
Student.objects.create(
    name="Rahim",
    email="rahim@example.com",
)

# READ
Student.objects.all()
Student.objects.get(pk=1)
Student.objects.filter(age__gte=18)

# UPDATE
Student.objects.filter(pk=1).update(
    is_active=False
)

# DELETE
Student.objects.filter(pk=1).delete()

# ORDERING
Student.objects.order_by("-created_at")

# LIMIT
Student.objects.all()[:10]

# EXISTS
Student.objects.filter(
    is_active=True
).exists()

# COUNT
Student.objects.count()

# OR CONDITION
```

```
from django.db.models import Q

Student.objects.filter(
    Q(age__lt=18) | Q(age__gt=60)
)

# F EXPRESSION
from django.db.models import F

Product.objects.update(
    stock=F("stock") - 1
)

# AGGREGATION
from django.db.models import Count

Author.objects.aggregate(
    total=Count("id")
)

# ANNOTATION
Author.objects.annotate(
    post_count=Count("posts")
)

# SELECT RELATED
Post.objects.select_related("author")

# PREFETCH RELATED
Author.objects.prefetch_related("posts")

# TRANSACTION
from django.db import transaction

with transaction.atomic():
    pass

# MIGRATIONS
# python manage.py makemigrations
# python manage.py migrate
# python manage.py showmigrations
```

Chapter 27: Learning Roadmap

Beginner Level

1. Learn tables, rows, columns, and primary keys.
2. Learn foreign keys.
3. Create Django models and fields.
4. Practice migrations.
5. Practice CRUD operations.
6. Write queries in the Django shell.

Intermediate Level

1. Learn field lookups and Q objects.
2. Practice ForeignKey, OneToOne, and ManyToMany relationships.
3. Learn F expressions.
4. Practice aggregation and annotation.
5. Use `select_related()` and `prefetch_related()`.
6. Learn validation and database constraints.
7. Practice transactions.

Advanced Level

1. Understand query plans and indexes.
2. Learn Subquery and Exists.
3. Practice bulk operations.
4. Create custom managers and QuerySets.
5. Learn multiple databases and routers.
6. Understand concurrency and transaction isolation.
7. Plan production migrations.
8. Monitor query performance.

Capstone Project: Student Course Management API

Models

- Student
- Course
- Instructor
- Enrollment

Requirements

- Student CRUD API
- Course CRUD API
- Student enrollment system
- Prevent duplicate enrollments
- Count students in each course
- List courses for each student
- Filtering and pagination
- Query optimization
- Transaction support
- Automated tests
- PostgreSQL configuration
- API documentation

Test Cases

- Duplicate email is rejected.
- Duplicate enrollment is prevented.
- Missing records are handled correctly.
- Unauthorized users cannot access protected data.
- N+1 queries are reduced.
- Failed transactions do not leave unwanted partial data.

Official Django Documentation

- Django 6.0: https://docs.djangoproject.com/en/6.0/
- Models: https://docs.djangoproject.com/en/6.0/topics/db/models/
- QuerySets: https://docs.djangoproject.com/en/6.0/ref/models/querysets/
- Query Expressions: https://docs.djangoproject.com/en/6.0/ref/models/expressions/
- Database Optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
- Transactions: https://docs.djangoproject.com/en/6.0/topics/db/transactions/
- Migrations: https://docs.djangoproject.com/en/6.0/topics/migrations/

Final Advice

Reading notes is not enough to master Django ORM.

Practice each example, inspect generated SQL, write tests, and measure query performance using realistic data.

Practice + Testing + Debugging = Strong Django ORM Skills